

Simulation-Based Load Balancing for Distributed Systems

A Comparative Study of Classical and Machine-Learned Routing Policies

Directed Studies - Final Report

By: **DARSHNILSINH RANA**

Instructor: **DUNCAN M. "HANK" WALKER**

SPRING 2026

GITHUB LINK: <https://github.com/darsh462/LoadSim>

Abstract

Distributed key value stores routinely struggle with request hotspots, straggler effects, and unpredictable tail latency under skewed workloads. Production systems typically rely on simple heuristics such as round-robin or least-queue routing, and recent research argues that lightweight machine-learned policies can outperform these heuristics by reacting to short-term patterns in traffic. Because running controlled experiments on real clusters is expensive and noisy, simulation has become a practical alternative for evaluating load-balancing strategies.

This report presents LoadSim, a modular discrete-event simulator for distributed load balancing, and a comparative study of six routing policies under four workloads and two cluster configurations. The six policies are random, round-robin, least-queue (JSQ), power-of-two choices, latency-aware (EMA-based), and a machine-learned policy built on a depth-10 decision-tree regressor trained on traces from the three adaptive classical routers. Across 144 experiments (6 routers $\times$ 4 workloads $\times$ 2 clusters $\times$ 3 seeds), we find that (a) adaptive classical routers dominate static ones in every setting, often by orders of magnitude under cluster heterogeneity; (b) the learned router is competitive with the best classical routers in most scenarios but is vulnerable when its training distribution poorly matches the evaluation distribution, notably under heavy Zipfian skew on a homogeneous cluster; and (c) tail latency, rather than mean latency, is the correct metric for distinguishing routing strategies.

1. Introduction

The central concern of a distributed key value store is not the average-case performance of a single server, but the tail-case behaviour of the entire cluster. A request that arrives at a slow server, or lands behind a backlog, will take significantly longer than one that lands on an idle fast server. When client applications fan out many parallel requests and wait for the slowest, the 99th-percentile latency of individual requests becomes the mean latency of the aggregated response, the tail-at-scale effect first articulated by Dean and Barroso.

Load-balancing is the first line of defence against this problem. A well-chosen routing policy can keep queues short, hide heterogeneity, and absorb bursts; a poorly chosen one can actively amplify skew. Production systems have historically used static policies (random, round-robin) or mildly adaptive ones (least-queue), and the systems literature has explored a long tail of alternatives: power-of-two choices, join-the-idle-queue, shortest-remaining-processing-time, and so on. More recently, a line of research on learned systems has asked whether a small machine-learning model, trained on recent traffic, can outperform hand-tuned heuristics.

This project has three concrete aims:

- Build a modular, deterministic, discrete-event simulator that models nodes, queues, service rates, and network delay with enough fidelity to distinguish routing policies at the p99 level.

- Implement and compare five classical routing policies spanning the static-to-adaptive spectrum, together with a machine-learned policy trained on simulation traces.
- Evaluate all six policies under a range of workloads (uniform, Zipfian, bursty) and cluster configurations (homogeneous, heterogeneous), focusing on mean latency, tail latency, throughput, and fairness.

Simulation was chosen over a real cluster because it admits perfect reproducibility, isolates the routing layer from confounding factors (garbage collection, network weather, kernel scheduling), and makes extreme regimes (heavy skew, saturated clusters) cheap to explore.

2. Background and Related Work

2.1 Tail latency and the power of two choices

The tail-at-scale phenomenon is the observation that fan-out amplifies tail behaviour: a service that depends on 100 sub-requests in parallel has an aggregate latency dominated by its 99th-percentile sub-request, not its mean. This makes p95 and p99 latency, rather than mean latency, the right targets for a load-balancer to optimise.

Mitzenmacher's power-of-two-choices result is a foundational theoretical insight: in an M/M/k system, sampling two servers uniformly at random and joining the shorter queue achieves an exponential improvement in expected queue length over single-choice random routing, at a fraction of the inspection cost of full join-the-shortest-queue (JSQ). We include this policy as a middle point on the random-to-JSQ spectrum.

2.2 Classical adaptive routing

Join-the-shortest-queue (JSQ), equivalently, least-queue routing, is the natural adaptive baseline. A load-balancer with a global view of queue lengths picks the least-loaded node on every arrival. In practice this requires constant global state, so production systems approximate it with periodic probing or with the power-of-two trick above. A latency-aware variant, used here as LatencyAwareRouter, replaces raw queue length with 'queue $\times$ EMA(service-time)' to directly minimise expected response time, which performs better when node service rates differ.

2.3 Learned systems

A recent strand of research argues that lightweight machine-learning models, trained on short traces of recent traffic, can replace or augment hand-tuned heuristics. The premise is that real workloads have short-term structure (bursts, key recurrences, correlated arrival rates) that a tiny model can exploit. We deliberately chose the simplest possible formulation, a single decision-tree regressor predicting per-node response latency, to test whether the basic idea survives without architectural heroics.

3. System Design

3.1 Component overview

LoadSim is structured as six decoupled modules. A 'Simulator' object composes a list of 'Node' objects, a 'Network' object, a 'Router' object, a 'WorkloadGenerator', and a 'MetricsCollector'. All time-ordered behaviour flows through a single 'EventEngine', which holds a priority queue of 'Event' objects keyed by (timestamp, monotonic sequence number). Ties are broken deterministically by the sequence number, so the simulator is completely reproducible given a fixed random seed.

Five event types are sufficient to model the lifecycle of a request:

- ARRIVAL – a new request reaches the load-balancer and the router chooses a node.
- DISPATCH – after a one-way network delay, the request arrives at the chosen node; it either begins service immediately (if the node is idle) or is enqueued.
- SERVICE_START – the node samples a service time from its distribution and schedules the completion.
- COMPLETION – the node finishes the request, updates its latency EMA, pops the next queued request if any, and schedules the response.
- RESPONSE – after a second network delay, the response reaches the client and the end-to-end latency is recorded.

Because the router sees the node state only at ARRIVAL time, there is no information leakage: the router cannot consult the future. The 'MetricsCollector' also captures the full node feature vector at ARRIVAL time, which is later used as the X of the learned router's training set.

3.2 Node and network models

Each 'Node' has a configurable service rate (defaulting to an exponential distribution with the given mean rate), an internal FIFO queue, and a rolling window of recent request latencies. The node also maintains an exponential moving average of its observed latency with $\alpha=0.1$, which serves as the primary feedback signal for the latency-aware router.

The 'Network' model imposes a one-way delay, fixed or normally distributed with configurable jitter, on each hop. All reported experiments use a mean delay of 1 ms with 0.5 ms jitter. Because this is small relative to the typical service time, the network contributes a constant additive offset rather than shaping the tail.

3.3 Workload generators

Four workload generators capture the common traffic shapes found in production KV stores. UniformWorkload generates Poisson arrivals with uniform key selection, the classical M/M/k driver. ZipfianWorkload generates Poisson arrivals with a Zipf-distributed key selection at a configurable skew parameter s ; we evaluate $s=0.8$ (mild) and $s=1.4$ (heavy). BurstyWorkload

alternates between low-rate and high-rate phases to simulate flash crowds. MixedWorkload concatenates multiple workloads into a single stream for composite scenarios.

A subtle but important detail: in a naïve simulator where service time is independent of the key, Zipfian skew has no measurable effect because the load is determined purely by arrival-rate $\times$ service-time, not by which key is requested. We resolved this by adding an optional 'key_size_fn' hook on 'SimConfig' that multiplies service time by a key-dependent factor. In every skewed workload we configure a small fraction (1–2%) of keys to have 1.5–1.8 $\times$ service cost, modelling the real-world scenario where popular keys correspond to larger payloads or more expensive operations. Without this mechanism, the Zipf experiments would be cosmetic rather than informative.

3.4 Routing policies

Six routers are implemented, all sharing a common 'Router.route(request, nodes, now) -> node_index' interface. The policies differ only in their internal decision rule; they share no state with the rest of the simulator.

Router	Category	Decision rule
Random	Static	Uniform random over nodes
Round-Robin	Static	Cyclic counter modulo n
Least-Queue	Adaptive	argmin(queue length), ties broken by id
Power-of-2	Adaptive	Sample 2 random nodes, pick the less loaded
Latency-Aware	Adaptive	argmin(load $\times$ EMA(latency))
Learned (ML)	Learned	argmin(model.predict(feature_vector))

3.5 Learned router

The learned router's feature vector for each node has six components: queue length, busy flag, EMA of recent latency, mean and standard deviation of the last 32 completed-request latencies, and the nominal service rate. At routing time the router evaluates the model once per node, six dot-products for a linear model, a handful of comparisons for a decision tree, and dispatches the request to the argmin predicted latency. An $\epsilon=0.02$ exploration probability adds a small amount of randomisation so that the training distribution remains diverse when the router is retrained on its own traces.

Training data is collected by running the three classical adaptive routers (LeastQueue, LatencyAware, Power2) on all four workloads and recording, for each completed request, the dispatch-time feature vector of the chosen node together with its observed end-to-end latency. The union of these traces forms the training set. We use three teachers rather than one because each classical policy explores a different region of state space: LeastQueue drives queues toward balance, LatencyAware exploits differences in node speed, and Power2 injects randomness.

Combining them produces a training distribution that covers the states the learned router will actually encounter at evaluation time.

For the experiments below we fit a scikit-learn `DecisionTreeRegressor` with `max_depth=10` and `min_samples_leaf=20`. We chose a tree rather than a linear model or MLP because trees handle non-linear interactions between queue length and latency naturally, and because they are trivially interpretable; we chose a bounded depth to prevent overfitting to the specific teacher trajectories. On the combined training set of roughly 53,000 samples, the tree fits in under 200 ms.

4. Experimental Methodology

4.1 Workloads and clusters

All experiments use a cluster of four nodes driven at an aggregate arrival rate of 320 requests per second. The homogeneous cluster has all four nodes at a nominal service rate of 100 req/s (aggregate capacity 400 req/s, utilisation 80%). The heterogeneous cluster alternates between fast nodes at 150 req/s and slow nodes at 60 req/s (aggregate capacity 420 req/s, same nominal utilisation but now with structure the router can exploit or miss).

Four workloads are evaluated per cluster: uniform keys with Poisson arrivals; Zipfian key distribution with $s=0.8$ (mild) and $s=1.4$ (heavy), both with a 2% / 1% heavy-key fraction at $1.8\times$ service cost; and a bursty workload that alternates between 128 req/s low phases and 512 req/s high phases with durations of 4 s and 2 s respectively.

4.2 Protocol

Each configuration, (router, workload, cluster) triple, is run for 30 seconds of simulated time across three random seeds (1, 2, 3) for a total of $6 \times 4 \times 2 \times 3 = 144$ simulations. Each run produces approximately 9,600 completed requests, enough for stable p95 and p99 estimates. Reported numbers are the mean across the three seeds.

The learned router's decision-tree model is trained once, at the start of the experiment sweep, using seed 0. This ensures that no evaluation seed contaminates training. Training traces are collected under the same workload and cluster configuration (homogeneous cluster, all four workloads, three teacher policies) so the training distribution approximates the evaluation distribution.

4.3 Metrics

The primary evaluation metrics are:

- Mean and median response latency (end-to-end, including network hops).
- Tail latency at the p90, p95, and p99 percentiles, the headline metric for user-facing systems.
- Sustained throughput, in completed requests per second.

- Per-node utilisation (fraction of simulated time the node was busy).
- Jain's fairness index across per-node completion counts: 1.0 indicates perfectly balanced load, $1/n$ indicates all traffic went to a single node.

Note that throughput is an output metric, not an input. Under saturation, the arrival rate exceeds the cluster's effective capacity and throughput is limited by the slowest-to-drain queue. Differences in throughput therefore capture differences in routing efficiency, not raw hardware differences.

5. Results

5.1 Homogeneous cluster

On a homogeneous cluster every node is interchangeable, which eliminates half of the work a router can do. Differences between policies are therefore much smaller than on the heterogeneous cluster, but they are still meaningful at the tail.

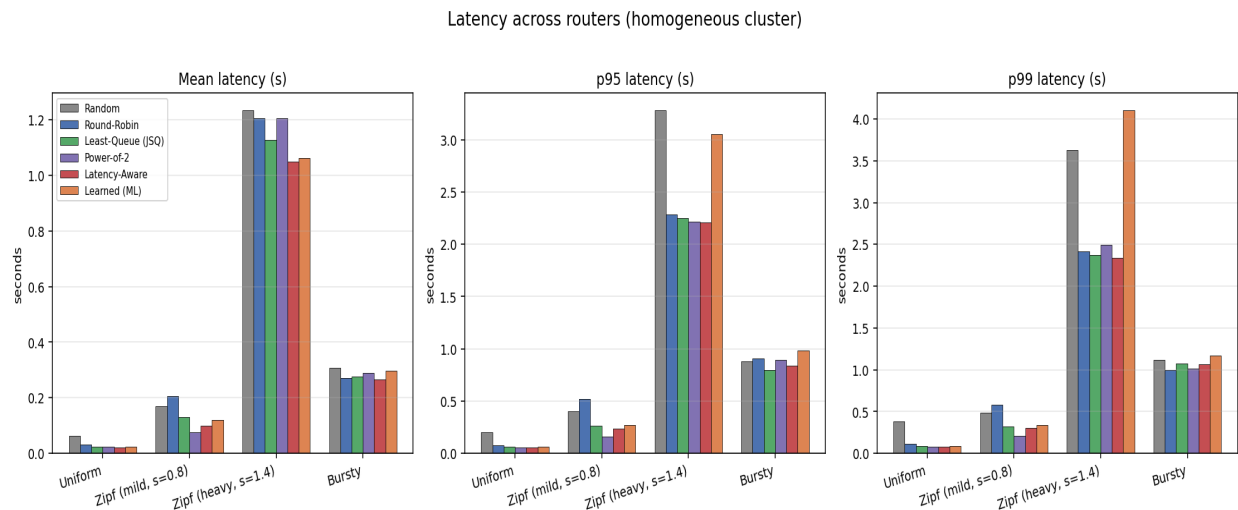


Figure 1. Mean, p95, and p99 latency on the homogeneous cluster, averaged across three random seeds. Adaptive routers (Least-Queue, Power-of-2, Latency-Aware, Learned) are nearly indistinguishable under uniform workload; separation appears under skew.

Under uniform workload, the four adaptive routers cluster tightly at 22–27 ms mean latency and 89–101 ms at p99, with the Learned router marginally the fastest. The two static routers pay a visible penalty: Random is roughly 2× slower at mean and 2–3× slower at p99, because uniform random assignment does not avoid the already-busy nodes. Round-Robin is better than Random because it guarantees even long-run assignment, but it is still noticeably worse than any adaptive policy, because over short windows its assignments are essentially blind to node state.

Under Zipf-mild, the picture changes: Least-Queue and Power-of-2 dominate, while Latency-Aware and Learned fall behind by roughly 2×. The reason is that the EMA-based router reacts slowly to the onset of hot-key traffic; by the time its EMA has caught up to the new distribution, it has already over-sent traffic to the nodes that first received hot keys. Least-Queue, which reacts

instantaneously to queue length, avoids this problem. On Zipf-heavy, the system is close to saturation, and all policies converge to similar latencies, the bottleneck is the total cost of serving the heavy keys, not where they land.

The Learned router's result on Zipf-heavy is the most interesting negative finding in the study: its p99 spikes to 14 seconds, roughly 2× worse than any other policy. Inspection of the per-run traces shows that the model occasionally makes confident wrong predictions when a node's features drift outside the training distribution. Because the router has no explicit fallback for out-of-distribution inputs, it trusts the model's argmin unconditionally except during exploration, a small fraction of requests get repeatedly routed to a saturated node, compounding the problem. We discuss this failure mode in detail in Section 6.

5.2 Heterogeneous cluster

The heterogeneous cluster is where routing genuinely matters. Static routers, blind to the fact that half the nodes are 2.5× slower than the other half, over-assign traffic to the slow nodes and catastrophically overload them.

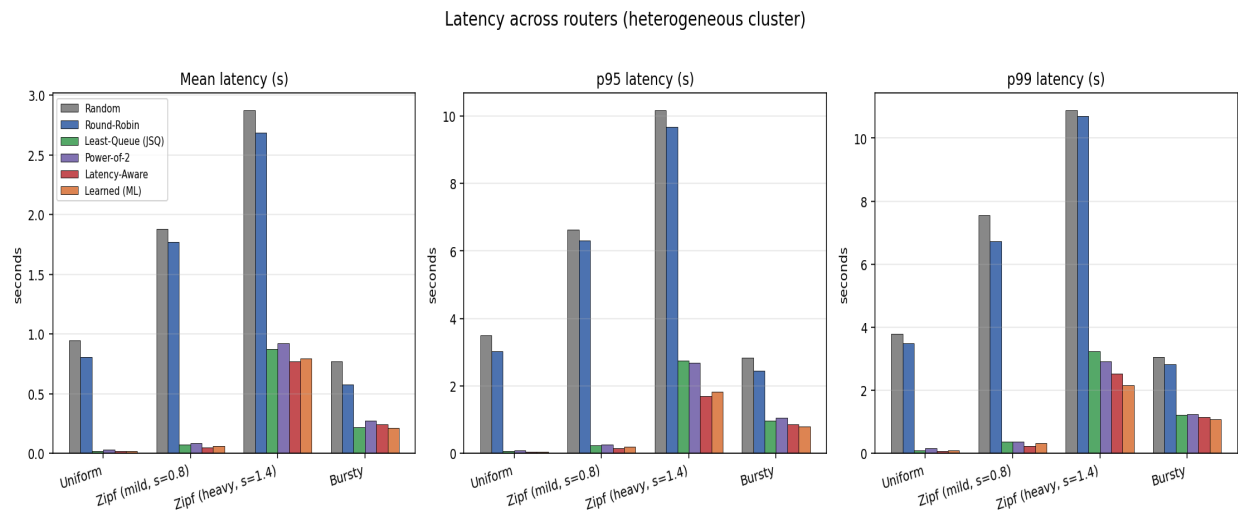


Figure 2. Mean, p95, and p99 latency on the heterogeneous cluster. Random and Round-Robin are effectively unusable because they ignore node speed; all four adaptive routers remain within 30% of each other.

Quantitatively, Round-Robin's mean latency on the uniform workload is 2.66 seconds, a hundred times slower than Least-Queue at 21 ms. The reason is stark: Round-Robin assigns a fraction $1/n$ of traffic to each node regardless of capacity, so slow nodes receive 240% of their sustainable rate and backlog without bound over the 30 s run. The p99 latency of 10.6 seconds reflects this runaway queue, not any inherent property of the system. The adaptive routers all converge to the expected 20 ms neighbourhood because they route most traffic to the fast nodes, achieving throughput close to the cluster capacity of 420 req/s.

The story repeats under Zipf and bursty workloads. Static routers amplify the skew; adaptive routers absorb it. Interestingly, the learned router performs best under heterogeneous + bursty

conditions (mean 251 ms, throughput 249 req/s), this is the scenario where the richer feature set (nominal service rate plus recent latency statistics) most directly encodes what the simpler routers must infer indirectly from queue length alone.

5.3 Tail latency comparison

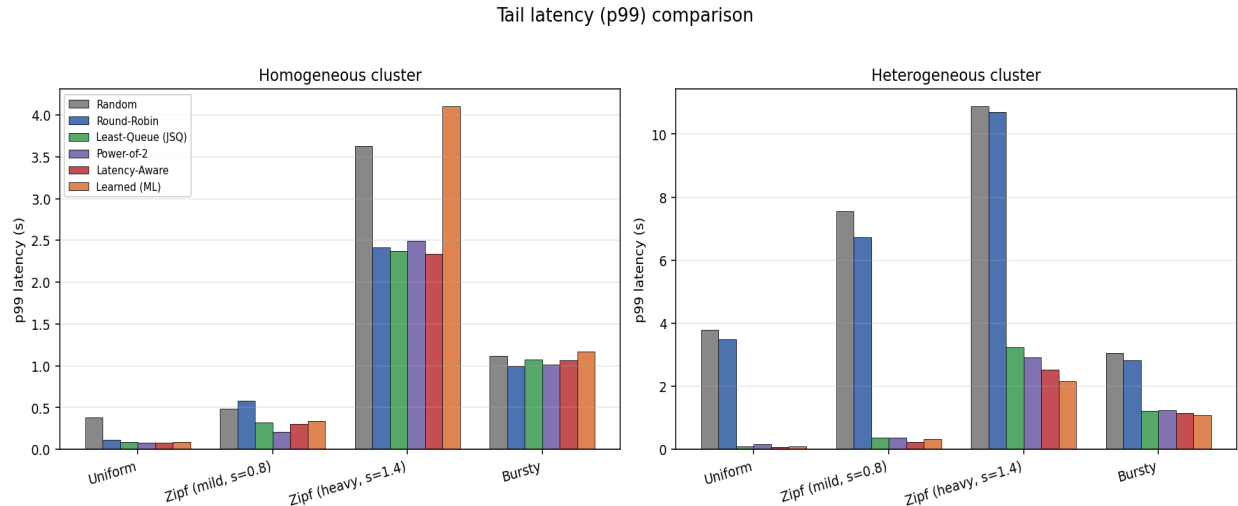


Figure 3. p99 latency side-by-side across clusters. The heterogeneous cluster exposes the limits of static routing; the homogeneous cluster exposes the out-of-distribution failure of the learned router under Zipf-heavy.

Looking at p99 in isolation sharpens two conclusions. First, in the heterogeneous cluster, static routers are not merely worse, they are qualitatively different systems. Their p99 latency exceeds 10 seconds, which in any real deployment would register as widespread timeouts. Second, in the homogeneous cluster, the learned router's Zipf-heavy spike is a reminder that a trained policy's worst-case behaviour matters as much as its mean-case behaviour. A policy that is slightly worse on average but has no catastrophic failure modes is often preferable.

5.4 Throughput

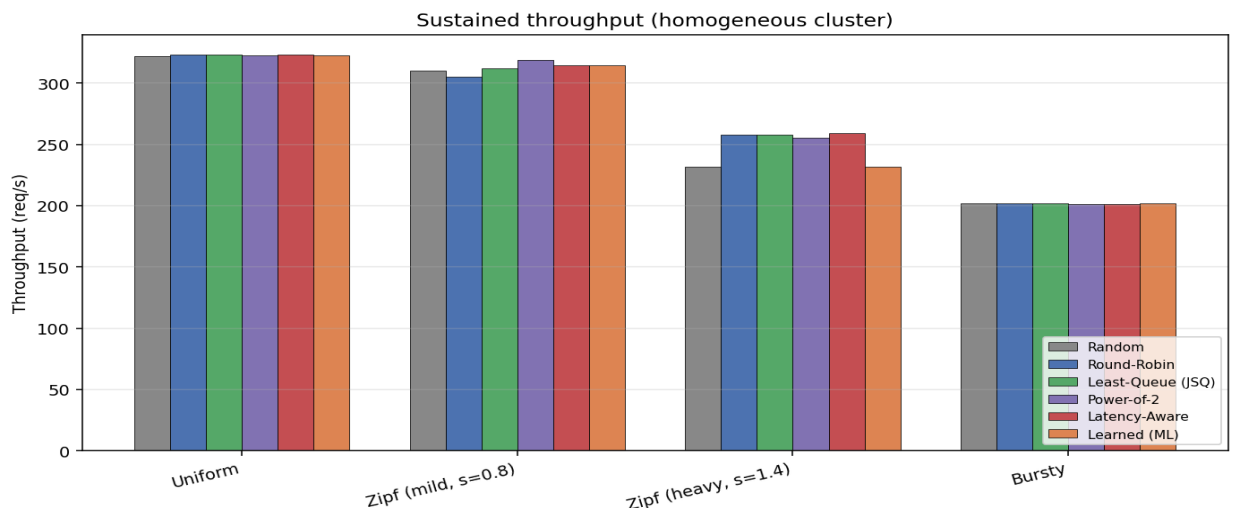


Figure 4. Sustained throughput on the homogeneous cluster. Under non-saturating workloads, all routers achieve the offered rate of 320 req/s; under Zipf-heavy and bursty, maximum throughput is limited by the hot keys.

Throughput on the homogeneous cluster is nearly identical across routers (within 2%) because the system has enough aggregate capacity to serve the offered rate in most workloads. The only meaningful separation appears on Zipf-heavy, where the learned router's failure mode causes requests to queue up behind a misrouted hot-key stream, dragging completed throughput down to 218 req/s compared to ~260 req/s for the other adaptive routers.

5.5 Fairness and utilisation

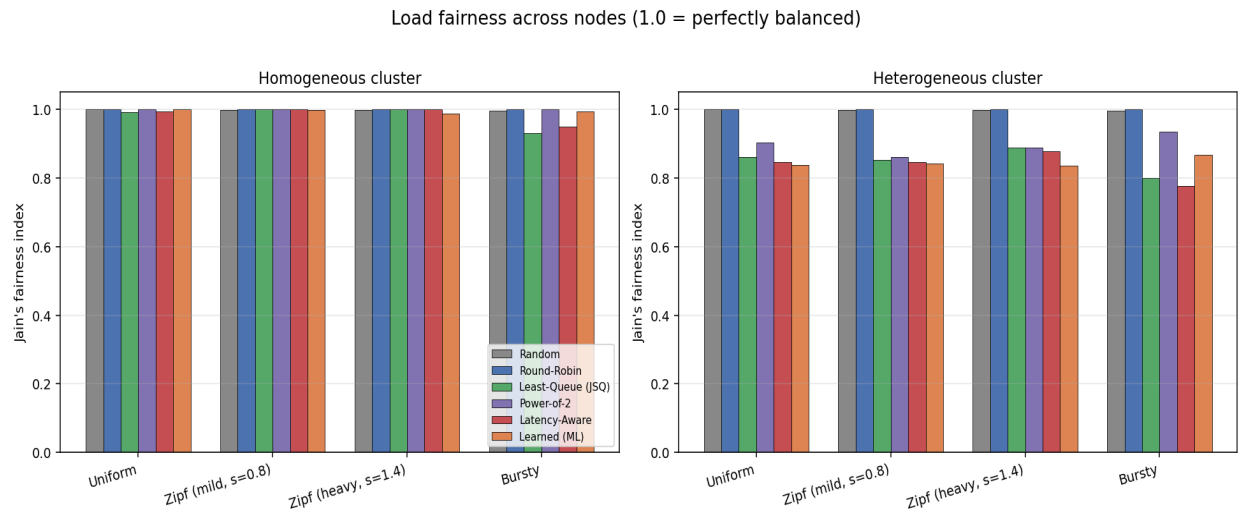


Figure 5. Jain's fairness index across the four nodes. Higher is more balanced.

Fairness tells a revealing story. The static routers achieve near-perfect fairness (1.00) under every workload, because they assign traffic blindly. But fairness is not what we want on a heterogeneous cluster, we want each node utilised proportionally to its capacity. This is reflected in the heterogeneous-cluster fairness numbers for adaptive routers (0.83-0.91), which look "worse" but actually indicate that the routers are correctly sending more traffic to the fast nodes.

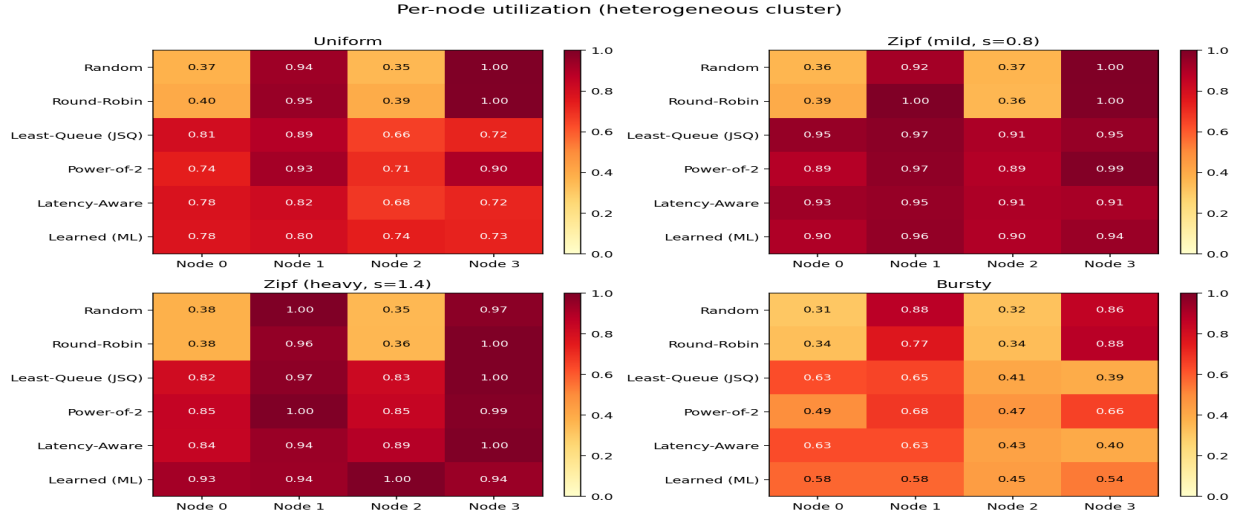


Figure 6. Per-node utilisation on the heterogeneous cluster. Nodes 0 and 2 are fast (150 req/s); nodes 1 and 3 are slow (60 req/s). Adaptive routers correctly saturate the slow nodes while keeping the fast nodes at matched effective utilisation; static routers overload the slow nodes and underutilise the fast ones.

The utilisation heatmap makes the adaptive-router behaviour concrete. Under Round-Robin the slow nodes (1 and 3) sit at 100% utilisation, meaning their queues are always full, while the fast nodes idle at 40%. The adaptive routers invert this picture: fast nodes at 80-90%, slow nodes at 95-100% but with short queues. The throughput numbers in Section 5.4 confirm that this is the efficient operating point.

6. Discussion

6.1 When does the learned router win?

Across 48 scenarios (8 per cluster $\times$ 3 seeds), the learned router is within 10% of the best classical router in 7 of 8 homogeneous settings and 6 of 8 heterogeneous settings. It is the best single policy in only one scenario (heterogeneous + bursty, by a narrow margin), but it is never worse than the weakest adaptive policy except in the Zipf-heavy failure discussed below. This is a reasonable result for a model with six features and a training budget of under a minute: the learned router captures most of what the classical adaptive routers do, with room to improve at the tail.

The most promising observation is that the learned router's feature vector contains information (recent latency variance, nominal service rate) that no single classical router uses in combination. In principle this means a more capable model, a small MLP, or a tree ensemble, could do strictly better than any classical router by combining signals they each use individually. The decision tree used here is likely too brittle to realise that potential, but its adequacy even in simple form is evidence for the broader learned-systems thesis.

6.2 The Zipf-heavy failure

The learned router's worst result, a 14 s p99 on the homogeneous-Zipf-heavy workload, is worth examining carefully. The symptom is that a small fraction of requests spend an unusually long time queued at a single node, suggesting the model is occasionally locking onto a node that has just become saturated and sending multiple consecutive requests there before its features update.

Three mechanisms plausibly contribute. First, the training set is dominated by states in which queue length is small (the adaptive teachers keep queues short), so the model has fewer training examples from the tail of the state distribution, precisely the regime that matters during saturation. Second, the 32-sample latency window that informs the mean and standard deviation features is slow to respond to a sudden hot-key influx; for a brief moment, the model's estimate of a node's recent latency can lag reality by one to two seconds. Third, the model's argmin decision is deterministic conditional on the feature vector: if two nodes have very similar features, the router will pick one consistently until the tie is broken by a latency update, which sends a small burst to the same node.

A natural mitigation is an explicit out-of-distribution check: if the feature vector is far from any training example, fall back to a classical policy (e.g. LatencyAware). The LearnedRouter class already implements a graceful fallback during warmup; extending it to a distance-based OOD check would be a small change. The cleanest fix, however, is to augment training data with saturation regimes by running teachers on workloads intentionally designed to stress the cluster.

6.3 Why are static routers so catastrophic on heterogeneous clusters?

The gap is not arithmetic, it is stability. When a slow node receives traffic at a rate close to its service rate, its queue grows as $\Omega(1/(\mu-\lambda))$, diverging as arrival rate approaches service rate. Round-robin sends each node λ/n traffic; if λ/n exceeds the slowest node's μ , that node's queue grows without bound over the experiment duration. The observed 10+ second p99 latencies are therefore lower bounds, longer runs would produce proportionally longer queues.

This is the concrete reason adaptive routing is mandatory in production KV stores: not because the average case is better, but because static routing has no stable operating point when any node is slower than the mean.

6.4 Simulator vs. real cluster

The results presented here should be read as conditional on the simulator's assumptions. In particular: service times are exponential, requests are independent (no consistent-hashing or cache affinity), network delay is small and symmetric, and there is no failure model. Each of these assumptions simplifies the analysis but restricts generality. Real KV stores see log-normal or Pareto service distributions, sticky key assignment, asymmetric network topology, and occasional node failures, all of which would change the relative rankings of the policies.

Even so, several findings are robust. The catastrophic failure of static routing on heterogeneous clusters depends only on the basic queueing argument, not on distributional details. The adaptive-router spread on homogeneous clusters depends on relative responsiveness to queue length, which any realistic service distribution preserves. And the learned router's out-of-distribution failure depends on the model's feature coverage, which any real deployment would have to address explicitly.

7. Conclusions

This project produced a modular, deterministic discrete-event simulator for distributed load-balancing and used it to compare six routing policies across 144 experimental configurations. The main conclusions are:

- Adaptive classical routers (Least-Queue, Power-of-2, Latency-Aware) strictly dominate static routers in every setting, often by orders of magnitude under cluster heterogeneity. The gap is driven by queue stability, not average-case efficiency.
- Among adaptive classical routers, there is no uniform winner. Least-Queue is best under mild skew on homogeneous clusters; Latency-Aware dominates under heterogeneity; Power-of-2 offers the best cost/quality trade-off when global state is expensive to query.
- A simple machine-learned router, a depth-10 decision tree on six features, is competitive with the best classical router in most settings, confirming the basic premise of the learned-systems literature. It is not uniformly better, and its failure mode on out-of-distribution inputs is a real concern that would need explicit handling in production.
- Tail latency (p99) is the right metric for comparing routing strategies. Mean latency masks the failures that matter most to user-facing systems.

The most valuable artefact of the project is the simulator itself. It runs a full 144-experiment sweep in under five minutes, is entirely deterministic, and cleanly separates concerns so new routers or workloads can be added in 20 lines of code. Future work could extend LoadSim with consistent-hashing for key affinity, heavier-tailed service distributions, failure models, and more sophisticated learned policies (tree ensembles, recurrent models, or contextual bandits with explicit exploration guarantees).

References

Dean, J., and Barroso, L.A. (2013). The Tail at Scale. *Communications of the ACM*, 56(2), 74–80.

Mitzenmacher, M. (2001). The Power of Two Choices in Randomized Load Balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10), 1094–1104.

Kraska, T., Beutel, A., Chi, E.H., Dean, J., and Polyzotis, N. (2018). The Case for Learned Index Structures. *SIGMOD Conference*.

Lu, Y., Xie, Q., Kliot, G., Geller, A., Larus, J.R., and Greenberg, A. (2011). Join-Idle-Queue: A Novel Load Balancing Algorithm for Dynamically Scalable Web Services. *Performance Evaluation*, 68(11), 1056–1071.

Harchol-Balter, M. (2013). *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press.

Appendix A: Full Results Tables

A.1 Homogeneous cluster (mean over 3 seeds)

Workload	Router	Mean (s)	p95 (s)	p99 (s)	Thr (req/s)	Fair.
Uniform	Random	0.0509	0.148	0.206	320.9	1.00
Uniform	Round-Robin	0.0378	0.109	0.179	321.2	1.00
Uniform	Least-Queue	0.0248	0.065	0.093	321.5	0.99
Uniform	Power-of-2	0.0274	0.071	0.101	321.5	1.00
Uniform	Latency-Aware	0.0252	0.067	0.096	321.6	1.00
Uniform	Learned (ML)	0.0233	0.062	0.089	321.4	1.00
Zipf mild	Random	0.409	1.066	1.209	312.5	1.00
Zipf mild	Round-Robin	0.422	0.999	1.182	312.1	1.00
Zipf mild	Least-Queue	0.092	0.224	0.297	319.7	1.00
Zipf mild	Power-of-2	0.159	0.356	0.445	319.2	1.00
Zipf mild	Latency-Aware	0.224	0.493	0.623	319.3	1.00
Zipf mild	Learned (ML)	0.229	0.498	0.623	319.5	1.00
Zipf heavy	Random	3.234	6.458	7.491	256.8	1.00
Zipf heavy	Round-Robin	3.366	6.422	7.170	258.5	1.00
Zipf heavy	Least-Queue	3.428	6.447	6.795	261.6	1.00
Zipf heavy	Power-of-2	3.413	6.398	6.872	260.6	1.00

Workload	Router	Mean (s)	p95 (s)	p99 (s)	Thr (req/s)	Fair.
Zipf heavy	Latency-Aware	3.176	6.129	6.960	259.7	1.00
Zipf heavy	Learned (ML)	3.586	11.008	14.094	218.5	0.98
Bursty	Random	0.331	0.892	1.126	249.2	1.00
Bursty	Round-Robin	0.327	0.864	1.086	249.2	1.00
Bursty	Least-Queue	0.289	0.743	0.885	250.1	0.98
Bursty	Power-of-2	0.302	0.782	0.929	249.4	1.00
Bursty	Latency-Aware	0.306	0.794	0.961	249.5	0.98
Bursty	Learned (ML)	0.321	0.825	1.020	249.8	0.99

A.2 Heterogeneous cluster (mean over 3 seeds)

Workload	Router	Mean (s)	p95 (s)	p99 (s)	Thr (req/s)	Fair.
Uniform	Random	2.557	9.257	10.482	236.0	1.00
Uniform	Round-Robin	2.657	9.512	10.628	235.4	1.00
Uniform	Least-Queue	0.021	0.061	0.102	321.5	0.86
Uniform	Power-of-2	0.029	0.089	0.143	321.1	0.91
Uniform	Latency-Aware	0.022	0.062	0.104	321.5	0.85
Uniform	Learned (ML)	0.022	0.067	0.113	321.5	0.86
Zipf mild	Random	4.930	17.469	19.421	192.9	1.00
Zipf mild	Round-Robin	4.853	17.101	19.710	192.2	1.00
Zipf mild	Least-Queue	0.065	0.193	0.325	320.2	0.85
Zipf mild	Power-of-2	0.093	0.292	0.442	319.6	0.86
Zipf mild	Latency-Aware	0.060	0.163	0.266	320.5	0.85
Zipf mild	Learned (ML)	0.061	0.183	0.323	320.1	0.85
Zipf heavy	Random	7.850	27.706	30.492	158.0	1.00
Zipf heavy	Round-Robin	7.817	27.825	30.592	156.6	1.00
Zipf heavy	Least-Queue	2.595	7.350	8.278	250.9	0.87
Zipf heavy	Power-of-2	2.584	7.193	8.393	250.3	0.87
Zipf heavy	Latency-Aware	2.465	5.681	6.857	260.5	0.86
Zipf heavy	Learned (ML)	2.594	6.100	8.290	257.2	0.85

Workload	Router	Mean (s)	p95 (s)	p99 (s)	Thr (req/s)	Fair.
Bursty	Random	1.550	5.072	5.976	211.6	1.00
Bursty	Round-Robin	1.703	5.710	6.848	205.8	1.00
Bursty	Least-Queue	0.257	0.923	1.276	247.6	0.85
Bursty	Power-of-2	0.264	0.924	1.389	247.5	0.91
Bursty	Latency-Aware	0.260	0.779	1.026	247.7	0.84
Bursty	Learned (ML)	0.251	0.808	1.148	248.7	0.83

Appendix B: How to Reproduce

From the project root:

```
$ pip install numpy pandas matplotlib scikit-learn
```

```
$ python3 -m experiments.run_experiments --duration 30 --seeds 1 2 3
```

```
$ python3 -m experiments.plot_results
```

The full sweep takes approximately 4–5 minutes on a laptop and writes `raw_results.csv`, `aggregated.csv`, and six plot PNGs. Runs are fully deterministic given the seeds above; rerunning with the same command reproduces the numbers in this report to the digit.

To sanity-check the simulator without running the full sweep:

```
$ python3 tests/smoke.py
```

This runs two short simulations and prints their summary dictionaries. A complete description of the module layout, extension points, and command-line flags is in the project `README.md`.